

Multithreading & ILP

(Micro-Architecture)

Review from Last Time

- Interest in **multiple-issue** because wanted to improve performance without affecting **uni-processor programming** model, i.e. simply by using parallelism at instruction/loop level in a program
- Taking advantage of ILP is conceptually simple, but design problems are amazingly complex in practice
- **How much ILP is possible with infinite resources ?**
- **How much ILP is achievable with finite resources/hazards.**
- Processors of last 25 years have the same basic structure and similar sustained issue rates (**3 to 4 instructions per clock**) as the 1st dynamically scheduled, multiple-issue processors announced in 1995
- Peak v. delivered performance gap increasing

Upper Limits to ILP: Assumptions

(Initial HW Model here; MIPS compilers)

Assumptions for **ideal/perfect machine** to start:

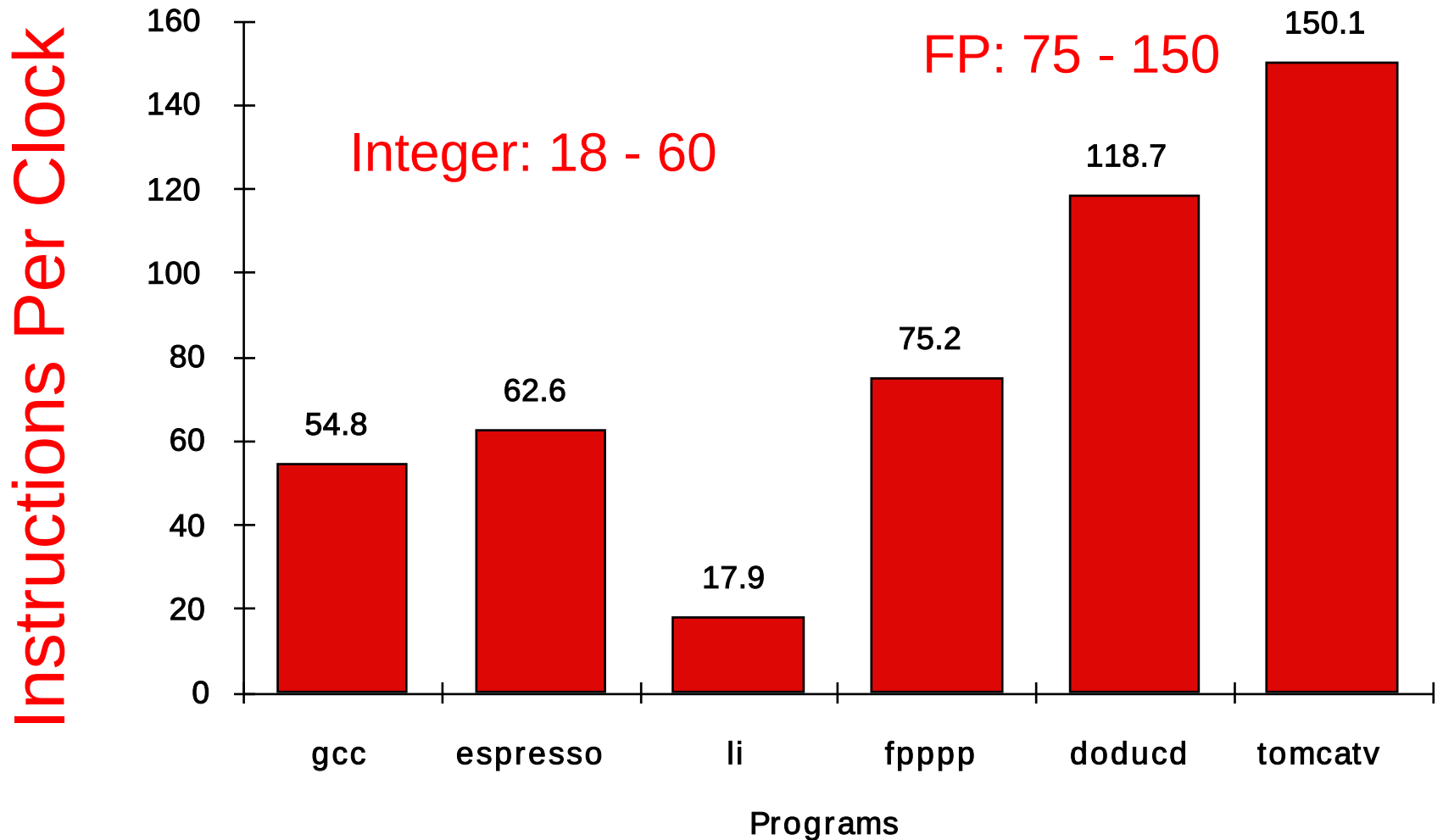
1. *Register renaming* – infinite virtual registers
=> all register WAW & WAR hazards are avoided
 2. *Branch prediction* – perfect; no misprediction;
perfect speculation
 3. Unbounded buffer of instructions available
- 1&3 eliminates all but RAW**

Also: perfect caches; 1 cycle latency for all instructions
(FP *, /); unlimited instructions issued/clock cycle;

Upper Limits to ILP: Comparison

	Perfect HW	Power 5
Instructions Issued per clock	Infinite	4
Instruction Window Size	Infinite	200
Renaming Registers	Infinite	48 integer + 40 Fl. Pt.
Branch Prediction	Perfect	2% to 6% misprediction (Tournament Branch Predictor)
Cache	Perfect	64K I, 32K D, 1.92MB L2, 36 MB L3

Upper Limits to ILP: Ideal Machine



How to Reach Upper ILP Limits?

- Practical limits for today due to hazards/resource constraints, can perhaps overcome via research/more HW (These are not laws of physics).
- Compiler and ISA advances could change ILP achieved
- Memory disambiguation: HW best
- Speculation: HW best
 - But very complicated to get Right
- Scheduling: SW can look ahead to schedule better

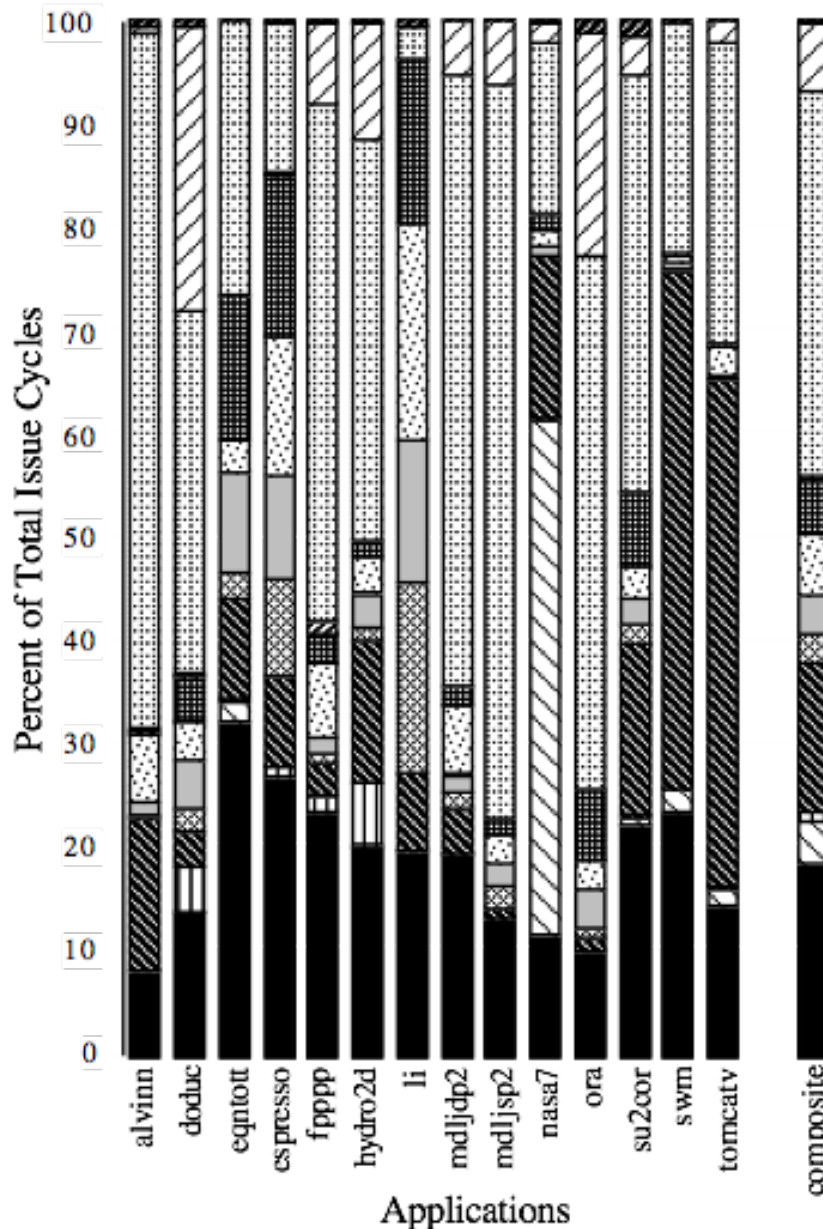
Practical Limits to ILP

- Doubling issue rates above today's 3-6 instructions per clock, say to 6 to 12 instructions, probably requires a processor to
 - issue 3 or 4 data memory accesses per cycle,
 - resolve 2 or 3 branches per cycle,
 - rename and access more than 20 registers per cycle, and
 - fetch 12 to 24 instructions per cycle.
- The **complexities** of implementing these capabilities is likely to **mean sacrifices in the maximum clock rate**
 - E.g, widest issue processor is the Itanium 2, but it also has the slowest clock rate, despite the fact that it consumes the most power!

Practical Limits to ILP

- Most techniques for increasing performance increase power consumption
- The key question is whether a technique is *energy efficient*: does it increase power consumption faster than it increases performance?
- **Multiple issue processors techniques all are energy inefficient:**
 1. Issuing multiple instructions incurs some overhead in logic that grows faster than the issue rate grows
 2. Growing gap between peak issue rates and sustained performance

Execution units lie idle



For an 8-way
superscalar.

Ex Units
is busy
only in this time

From: Tullsen,
Eggers, and Levy,
“Simultaneous
Multithreading:
Maximizing On-chip
Parallelism, ISCA
1995.

Recall: How to Handle Data Dependences

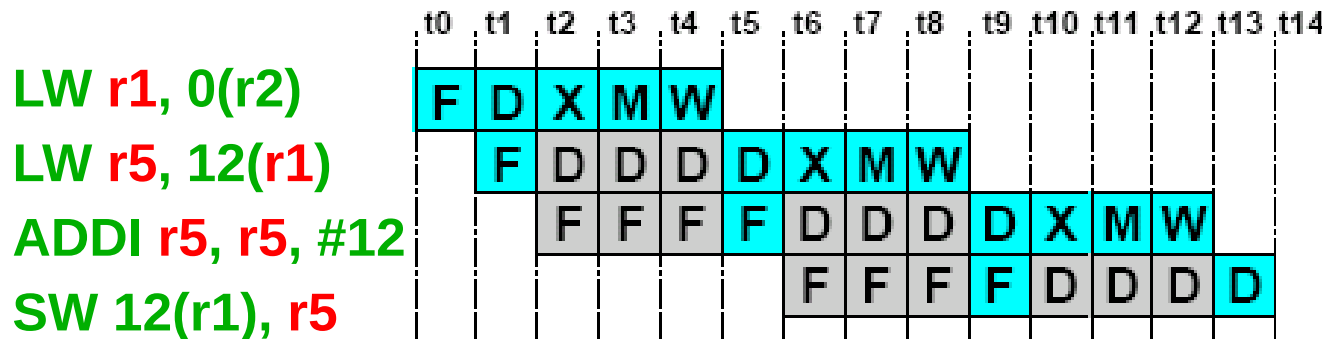
- Five fundamental ways of handling data dependences
 - **Detect and wait** until value is available in register file
 - **Detect and forward/bypass** data to dependent instruction
 - **Detect and eliminate** the dependence at the software level
 - No need for the hardware to detect dependence
 - **Predict** the needed value(s), execute “speculatively”, and **verify**
 - **Do something else** (fine-grained multithreading)
 - No need to detect

How to Handle Control Dependences

- Critical to keep the pipeline full with correct sequence of dynamic instructions.
- **Potential solutions if the instruction is a control-flow instruction:**
 - Stall the pipeline until we know the next fetch address
 - Guess the next fetch address (branch prediction)
 - Employ delayed branching (branch delay slot)
 - Fetch from both possible paths (if you know the addresses of both possible paths) (multipath execution)
 - Do something else (fine-grained multithreading)

Dealing with Pipeline Hazards

- Each instruction may depend on the previous
 - Without **bypassing**, need interlocks/stalls



- Bypassing (Superscalar Out of Order Ex/ Static Compiler Re-ordering)** cannot completely eliminate interlocks or delay slots/stalls
- How can we avoid the stalls due to dependencies between instructions in a pipeline?

Multithreading

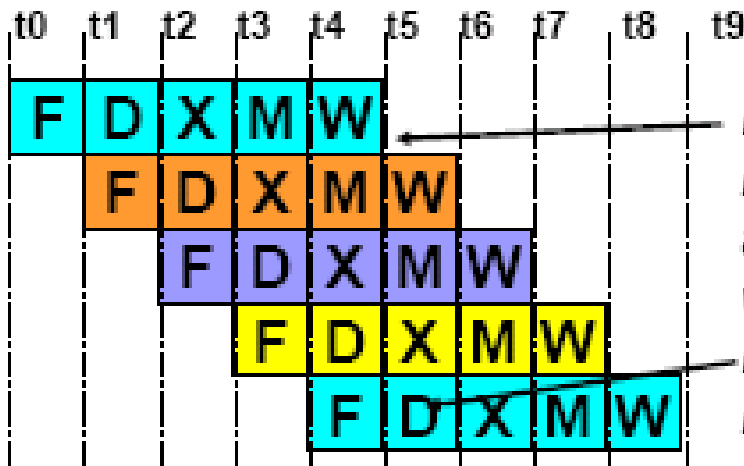
- How can we guarantee no dependencies between instructions in a pipeline?
 - One way is to interleave execution of instructions from different program threads on same pipeline
- Interleave 4 threads, T1-T4, on non-bypassed 5-stage pipe

Different Context

Same Context

T1: LW r1, 0(r2)
 T2: ADD r7, r1, r4
 T3: XORI r5, r4, #12
 T4: SW 0(r7), r5
 T1: LW r5, 12(r1)

Penalty:
Context Switching



Last instruction
 in a thread
 always completes
 writeback before
 next instruction
 in same thread
 reads regfile

From ILP to.....

Thread Level Parallelism (TLP)

- ILP exploits **implicit parallel operations** within a loop or straight-line code segment
- **ILP is limited in a program** particularly due to limited window size, data dependency and branches
- **TLP** explicitly represented by the use of **multiple threads of execution** that are explicitly parallel
- Goal: Use **multiple instruction streams** to improve
 1. Throughput of computers that run many programs
 2. Execution time of multi-threaded programs
- **TLP could be more cost-effective to exploit than ILP**
- Very suitable for multicore processors

What is a **Thread** in TLP

- There can be much higher level parallelism in some applications
(e.g., Database or Scientific codes)
- **Explicit Thread Level Parallelism** or **Data Level Parallelism**
- **Thread**: process with own instructions and data
 - Thread may be a process part of a parallel program comprising of multiple processes, or it may be an independent program
 - Each thread has all **the state** (instructions, data, PC, register state, and so on) necessary to allow it to execute
- **Data Level Parallelism**: Perform identical operations on data, and lots of data. **Single Instruction Stream Multiple Data Stream (SIMD)**

Multithreading in HW avoids Memory Latency

- In OS, processors switch to another context on I/O operation => **Software Multithreading**, Multiprogramming, etc. An O/S function. **Large overhead! Why? Context switch using Stack in Main Memory**
- **Why not context switch on a cache miss? => Hardware multithreading.**
- Can we afford that overhead with context switching? => **Need changes in HW architecture to avoid stack operations.** How to achieve it?
- Have many contexts can be **CPU resident** (not memory resident) by having **separate PCs and registers for each thread.** No need to store them in stack on context switching.

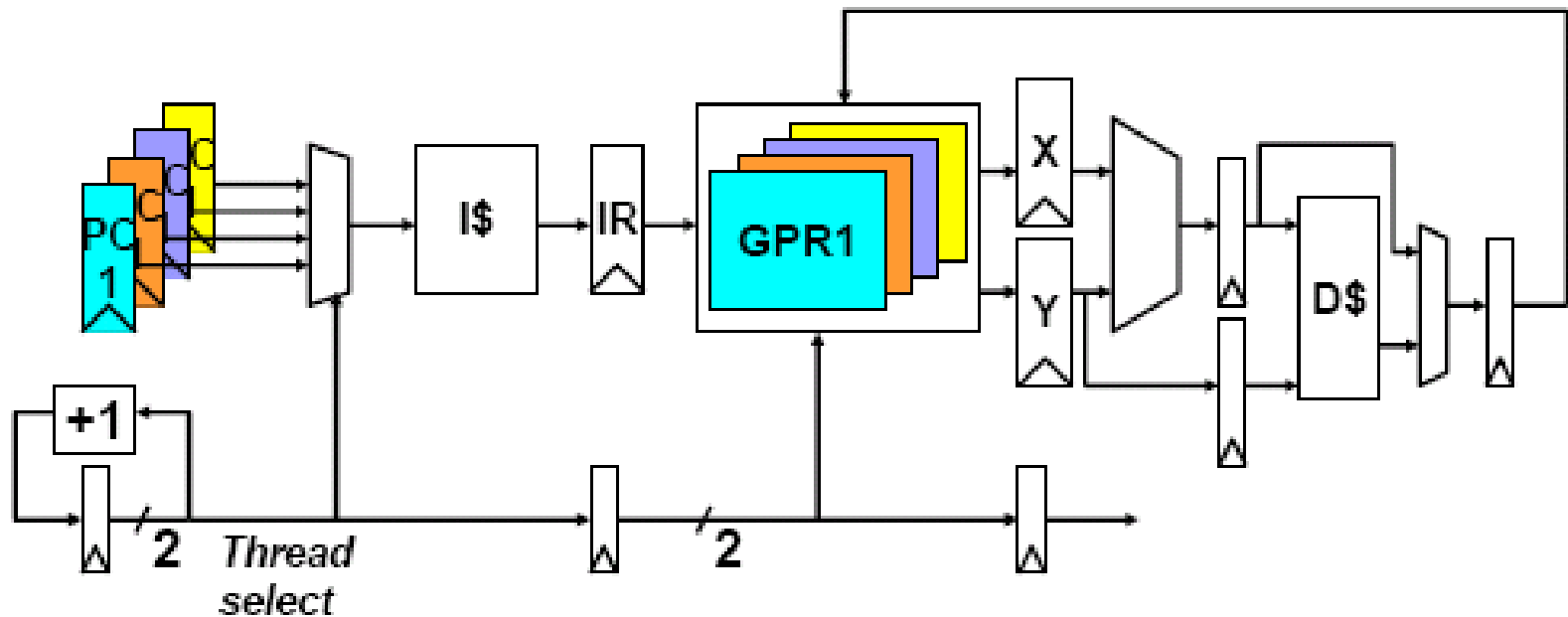
CDC 6600 Peripheral Processors (Cray, 1965)

- **First multithreaded hardware**
- **10 “virtual” I/O processors**
- **Fixed interleave** on simple pipeline
- **Each processor executes one instruction every 1000ns**
- **Accumulator-based instruction set to reduce processor state**

Hardware Multithreading

- We need to develop a hardware multithreading technique because switching between threads in software is very time-consuming (**Why?**), **Ex: Multitasking**
- Develop **multiple PCs and register sets** on the CPU so that thread switching can occur without having to store the register contents in main memory (**like it is done using stack**, for **context switching**).
- Several threads reside in the CPU simultaneously, and execution **switches between the threads on main memory access/cache miss**.
- How about both multiprocessors and multithreading on a chip? => **Network Processor**

Simple Multithreaded Pipeline

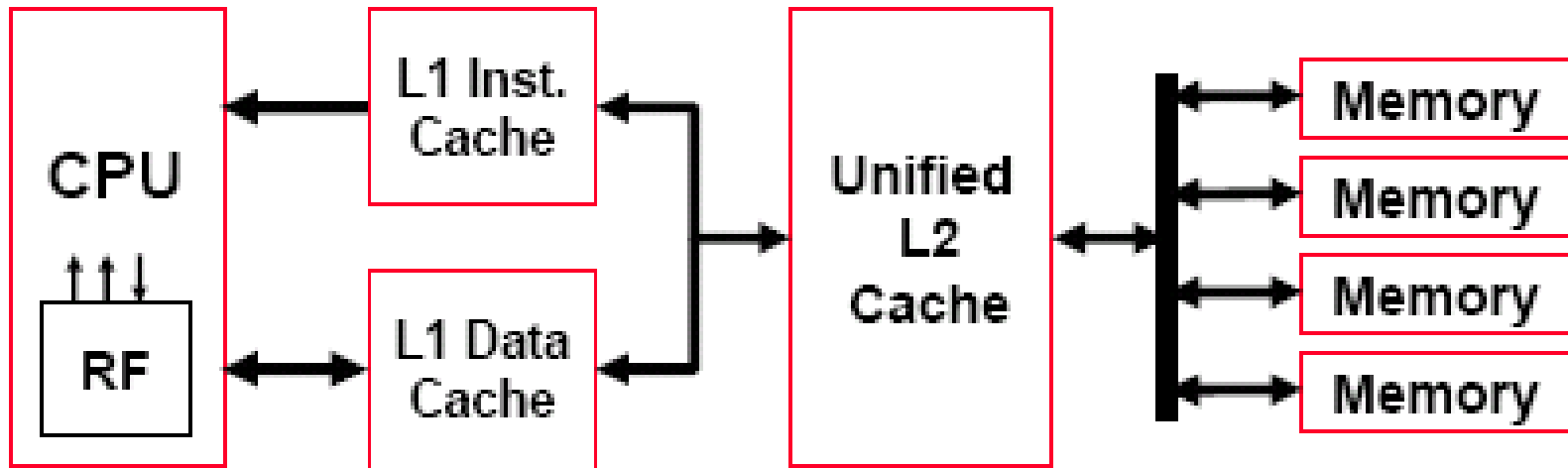


- **Have to carry “thread select”, down pipeline to ensure correct state bits read/written at each pipe stage**
- This is similar to carrying the control bits in a pipelined design across the stages

HW Multithreading Implications

- Appears to software (including OS) as **multiple faster CPUs**
- Each **thread** requires its own **user state (context)**
 - **GPRs**
 - **PC**
- Also, needs own OS **control state**
 - **Virtual memory page table base register**
 - **Exception handling registers**

Multithreading Design Choices



- **Context switch** to another thread every cycle, or on hazard or L1 miss or L2 miss or network request
- Per-thread **state** and **context-switch overhead**
- **Interactions between threads**, where in memory hierarchy

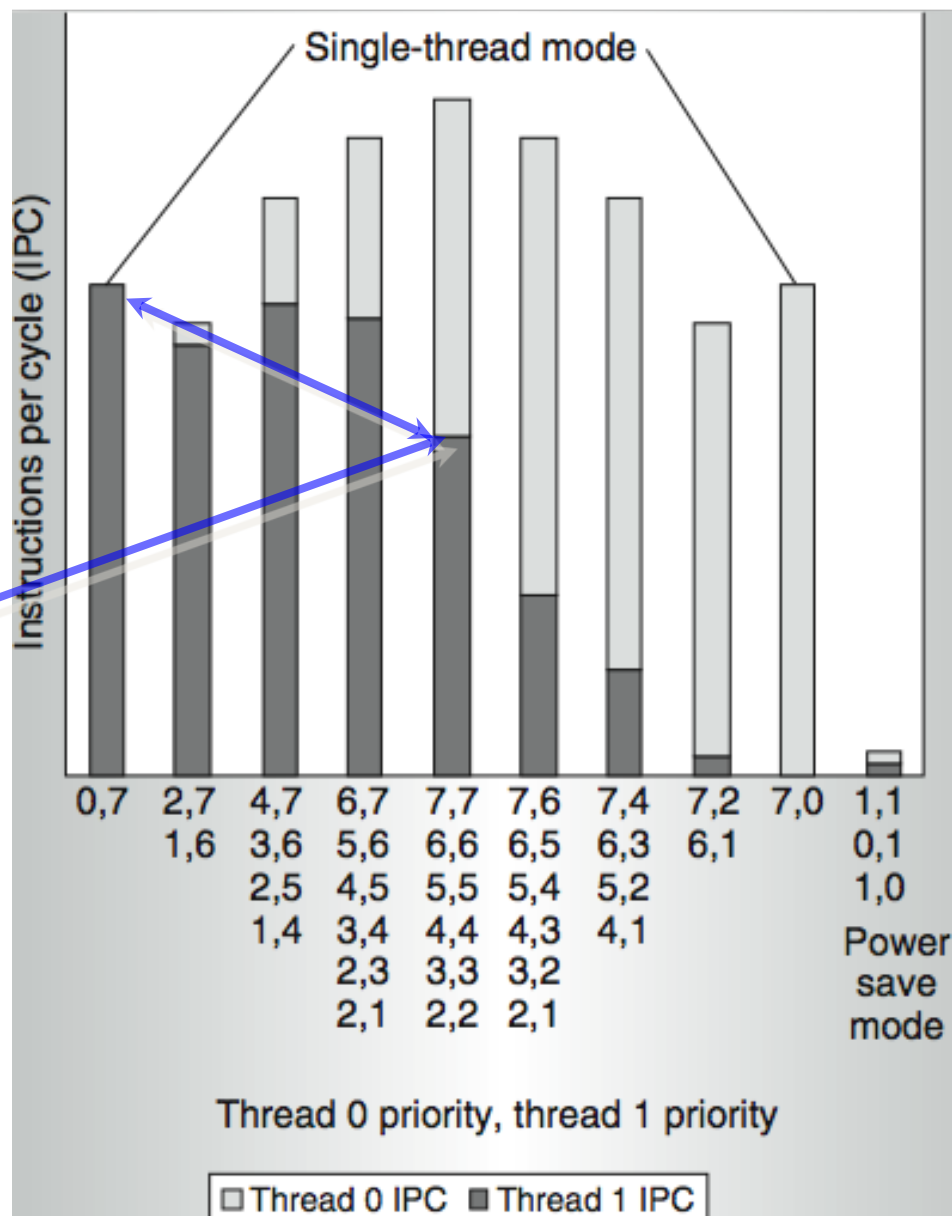
Thread Scheduling Policies

- **Fixed interleave** (*CDC 6600 PPU's, 1965*)
 - each of N threads executes one instruction every N cycles
 - if thread not ready to go in its slot, insert **pipeline bubble**
- **Software-controlled interleave** (*TI ASC PPU's, 1971*)
 - OS allocates S pipeline slots amongst N threads
 - hardware performs fixed interleave over S slots, executing whichever thread is in that slot
 - **No pipeline bubble**
- **Hardware-controlled thread scheduling** (*HEP, 1982*)
 - hardware keeps track of which threads are ready to go
 - picks next thread to execute based on **hardware priority scheme**

Power 5 thread performance ...

Relative priority of each thread controllable in hardware.

For balanced operation, both threads run slower than if they “owned” the machine.



Multithreaded HW Execution

- Multithreading: multiple threads to share the functional units of **1 processor** via overlapping
 - Processor must **keep independent state of each thread** e.g., a separate copy of register file, a separate PC, and for running independent programs, **a separate page table**
 - Memory shared through the **virtual memory** mechanisms, which **already support multiple processes** (i.e. Multitasking)
 - HW for fast thread switch; much faster than full process switch $\approx$ 100s vs 1000s of clocks
- When switch? (**Grain of Multithreading**)
 - Alternate instruction per thread (**fine grain**)
 - When a thread is stalled, perhaps for a cache miss, another thread can be executed (**coarse grain**)

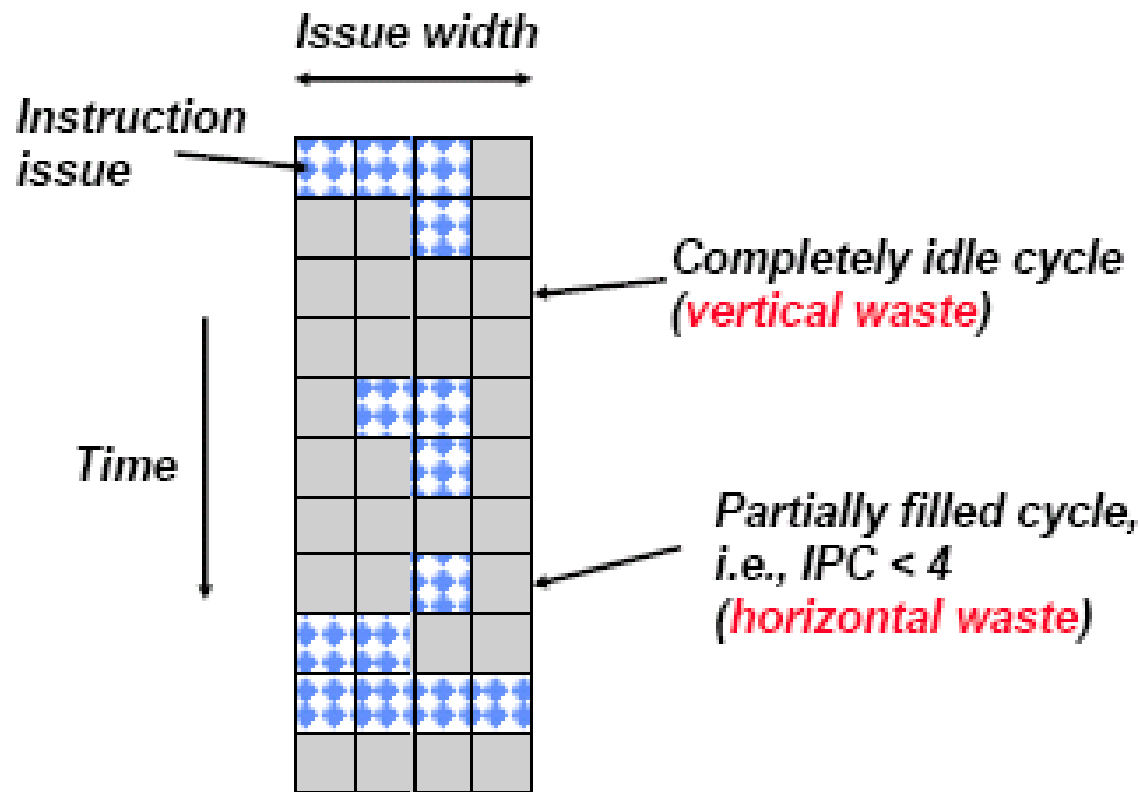
What is “Grain” in Multithreading?

- **So far assumed fine-grained multithreading**
 - CPU switches every cycle to a different thread
 - *When does this make sense?*
- **Coarse-grained multithreading**
 - CPU switches every few cycles to a different thread
 - *When does this make sense?*

When to do Fine Grain vs. Coarse Grain Multithreading

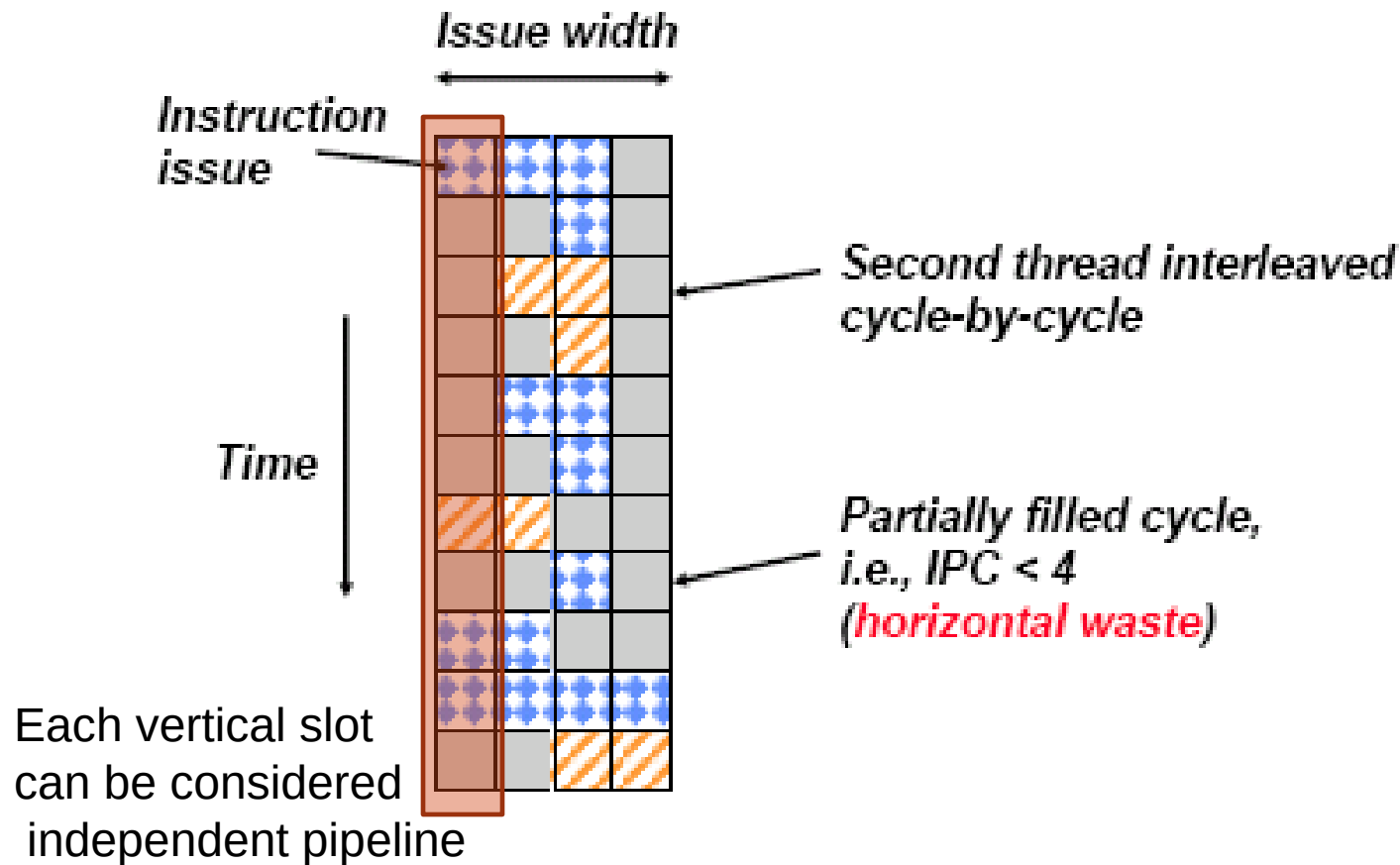
- Tera MTA designed for supercomputing applications with **large data sets** and **low locality**
 - No data cache
 - **Many parallel threads** needed to hide large memory latency
- Other applications are more cache friendly
 - Few pipeline bubbles when cache getting hits
 - **Just add a few threads** to hide **occasional cache miss latencies**
 - Swap threads on cache misses

Superscalar Efficiency



- Why horizontal waste?
- Why vertical waste?

Vertical Multithreading



- **Cycle-by-cycle interleaving of second thread removes vertical waste**

Fine-Grained Multithreading

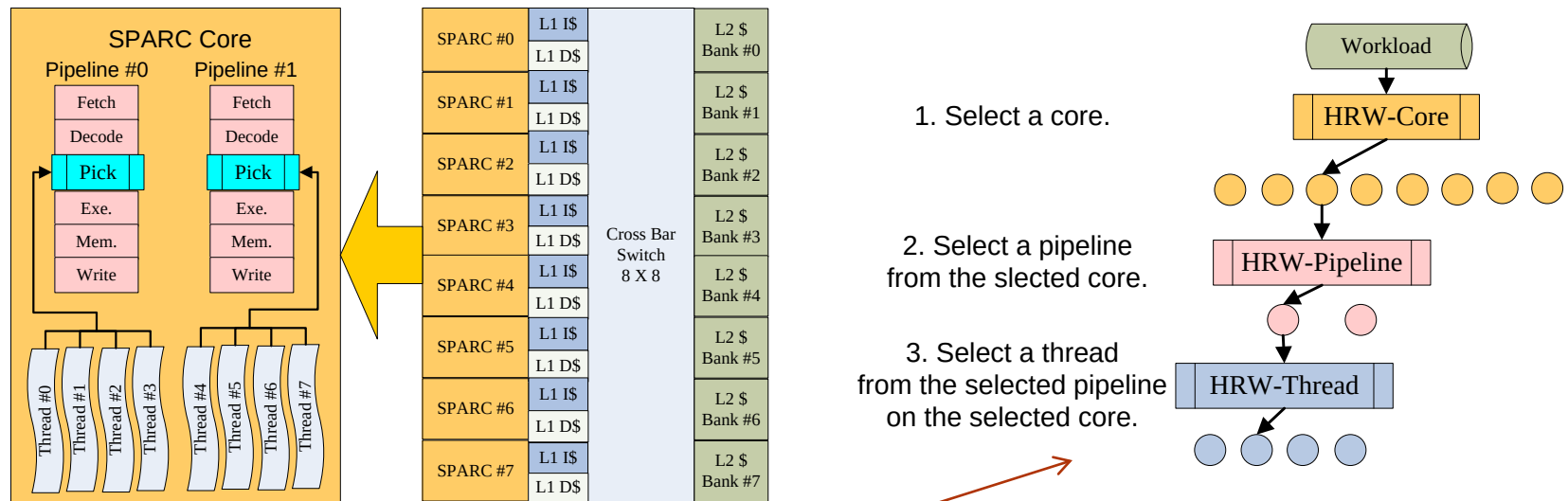
- Switches between threads **on each instruction**, causing the execution of multiples threads to be interleaved
- Usually done in a **round-robin fashion**, skipping any stalled threads
- CPU must be able to **switch threads every clock**
- **Advantage** is it can hide **both short and long stalls**, since instructions from other threads executed when one thread stalls
- **Disadvantage** is it slows down execution of individual threads, since a **thread ready to execute without stalls will be delayed** by instructions from other threads
- Used on **Sun's Niagara (will see later)**

Course-Grained Multithreading

- Switches threads **only on costly stalls, such as L2 cache misses**
- **Advantages**
 - Relieves need to have very fast thread-switching
 - **Doesn't slow down a thread**, since instructions from other threads issued only when the thread encounters a costly stall
- **Disadvantage** is hard to overcome throughput losses from **shorter stalls**
 - Since CPU issues instructions from 1 thread, when a stall occurs, the pipeline must be emptied or frozen
- Because of this **start-up overhead**, coarse-grained multithreading is **better for reducing penalty of high cost stalls**, where pipeline refill $\ll$ stall time
- **Used in IBM AS/400, GPU**

Sun Niagara2 (Multithreaded processor)

- Sun T5120 Niagara 2 – **Fine-grained multithreading**
 - 8 cores on chip each with 2 pipelines/core
 - 4 HW threads/pipeline => **64 threads in parallel**



Thread Scheduling: First schedule on **different cores** in a **round-robin manner** to avoid resource contention **then on different pipeline** and **finally within the same pipeline**

Sun SPARC Multithreaded Pipeline



Kongetira et al., "Niagara: A 32-Way Multithreaded Sparc Processor," IEEE Micro 2005.

Fine-Grained Multithreading (I)

- Idea: Hardware has multiple thread contexts (PC+registers). Each cycle, fetch engine fetches from a different thread.
 - By the time the fetched branch/instruction resolves, no instruction is fetched from the same thread
 - Branch/instruction resolution latency overlapped with execution of other threads' instructions
- Tolerates the control and data dependency latencies by overlapping the latency with useful work from other threads
- Improves pipeline utilization by taking advantage of multiple threads

Fine-Grained Multithreading (II)

- + No logic needed for handling control and data dependences within a thread
 - Single thread performance suffers
 - Extra logic for keeping thread contexts
 - Does not overlap latency if not enough threads to cover the whole pipeline
-
- Thornton, “Parallel Operation in the Control Data 6600,” AFIPS 1964.
 - Smith, “A pipelined, shared resource MIMD computer,” ICPP 1978.

Fine-grained Multithreading (III)

- Advantages

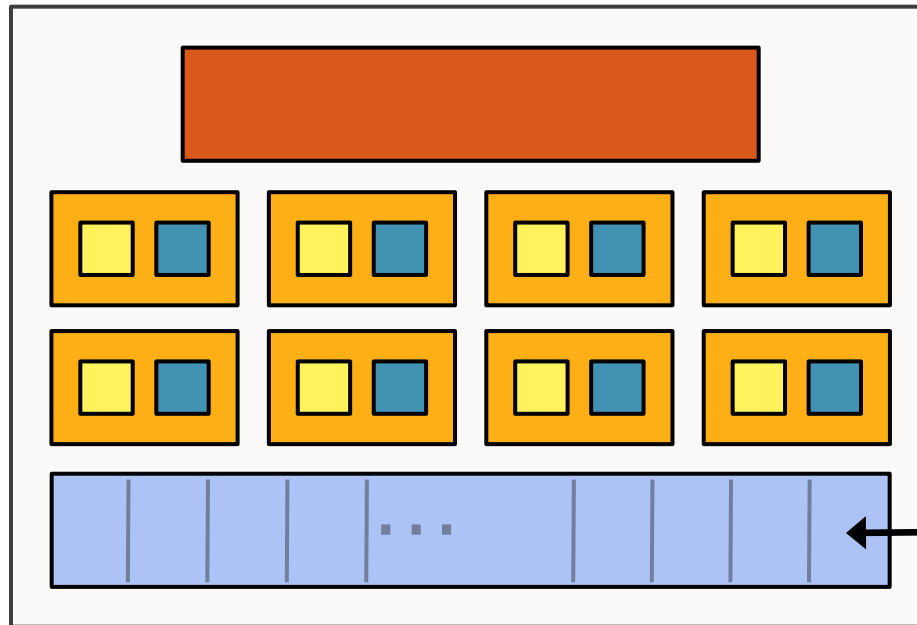
- + No need for dependency checking between instructions
(only one instruction in pipeline from a single thread)
- + No need for branch prediction logic
- + Otherwise-bubble cycles used for executing useful instructions from different threads
- + Improved system throughput, latency tolerance, utilization

- Disadvantages

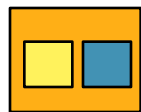
- Extra hardware complexity: multiple hardware contexts (PCs, register files, ...), thread selection logic
- Reduced single thread performance (one instruction fetched every N cycles from the same thread)
- Resource contention between threads in caches and memory
- Some dependency checking logic *between* threads remains (load/store)

Modern GPUs are FGMT Machines....

NVIDIA GeForce GTX 285 “core”



64 KB of storage
for thread contexts
(registers)



= data-parallel (SIMD) **func. unit**,
control shared across 8 units

■ = multiply-add
■ = multiply

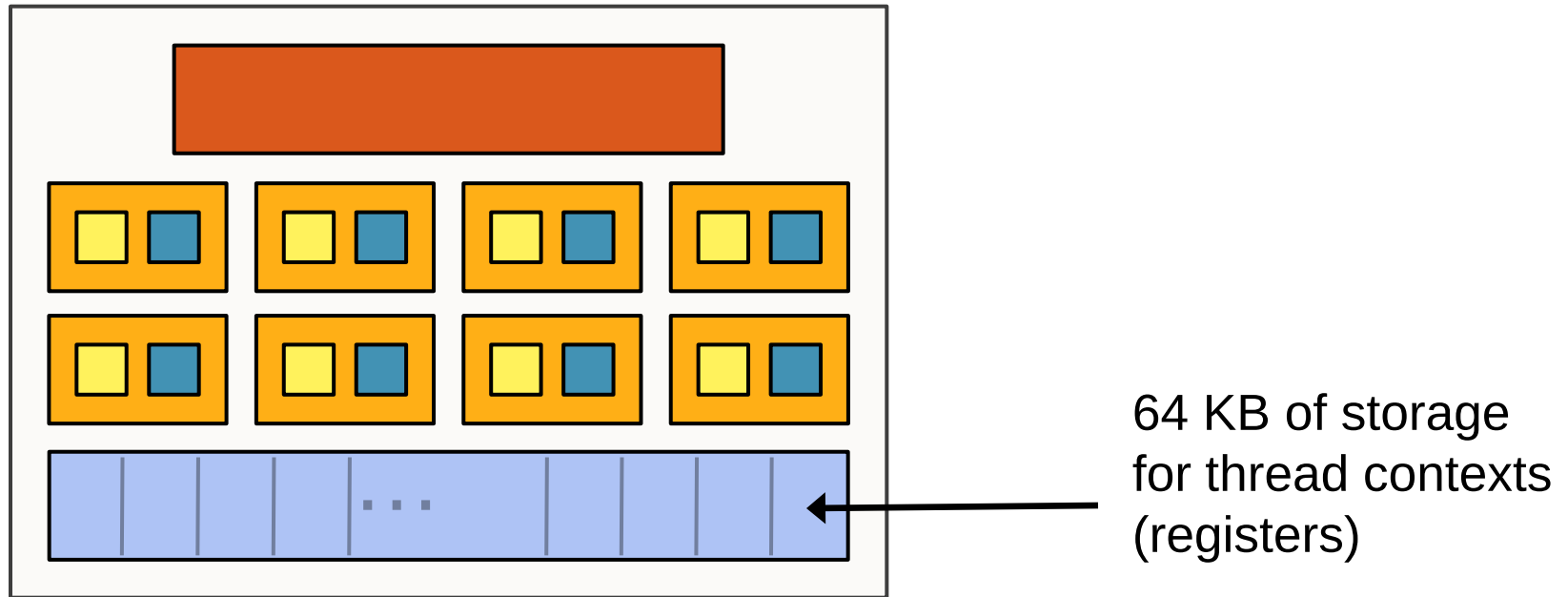


= instruction stream decode



= execution context storage

NVIDIA GeForce GTX 285 “core”



- Groups of 32 threads share instruction stream (**each group is a Warp**): they execute the same instruction on different data and **run in parallel**
- **Up to 32 warps are time interleaved in an FGMT manner**
- Up to 1024 thread contexts can be stored ($32 \times 32 = 1024$)

Do both ILP and TLP?

- TLP and ILP exploit two different kinds of parallel structure in a program
- Could a processor oriented at ILP to exploit TLP?
 - functional units are often idle in data path designed for ILP because of either stalls or dependences in the code
- Could the TLP be used as a source of independent instructions that might keep the processor busy during stalls?
- Could TLP be used to employ the functional units that would otherwise lie idle when insufficient ILP exists?

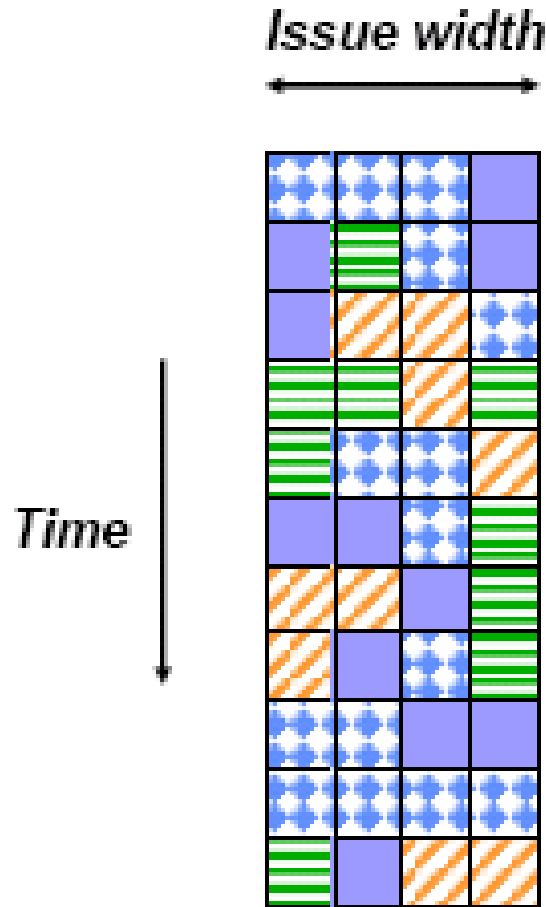
ILP ~ TLP competition

Processor	Micro architecture	Fetch / Issue / Execute	FU	Clock Rate (GHz)	Transis-tors Die size	Power
Intel Pentium 4 Extreme	Speculative dynamically scheduled; deeply pipelined; SMT	3/3/4	7 int. 1 FP	3.8	125 M 122 mm ²	115 W
AMD Athlon 64 FX-57	Speculative dynamically scheduled	3/3/4	6 int. 3 FP	2.8	114 M 115 mm ²	104 W
IBM Power5 (1 CPU only)	Speculative dynamically scheduled; SMT; 2 CPU cores/chip	8/4/8	6 int. 2 FP	1.9	200 M 300 mm ² (est.)	80W (est.)
Intel Itanium 2	Statically scheduled VLIW-style	6/5/11	9 int. 2 FP	1.6	592 M 423 mm ²	130 W

Why ILP + TLP

- **Itanium architecture** does **not** represent a significant breakthrough in scaling ILP or in avoiding the problems of complexity and power consumption
- **Instead of pursuing more ILP, architects are increasingly focusing on TLP implemented with multiprocessors**
- In 2000, IBM announced the 1st commercial single-chip, general-purpose multiprocessor, the Power4, which contains 2x Power3 processors **and an integrated L2 cache**
 - Since then, Sun Microsystems, AMD, and Intel have switch to a focus on single-chip **multiprocessors rather than more aggressive uniprocessors.**
- **Right balance of ILP and TLP is unclear today**

Ideal Multithreading for Superscalar



- Interleave multiple threads to multiple issue slots
with no restrictions dynamically

Simultaneous Multi-threading ...

One thread, 8 units

Cycle M M FX FX FP FP BR CC

1								
2								
3								
4								
5								
6								
7								
8								
9								

Two threads, 8 units

Cycle M M FX FX FP FP BR CC

1								
2								
3								
4								
5								
6								
7								
8								
9								

M = Load/Store, FX = Fixed Point, FP = Floating Point, BR = Branch, CC = Condition Codes

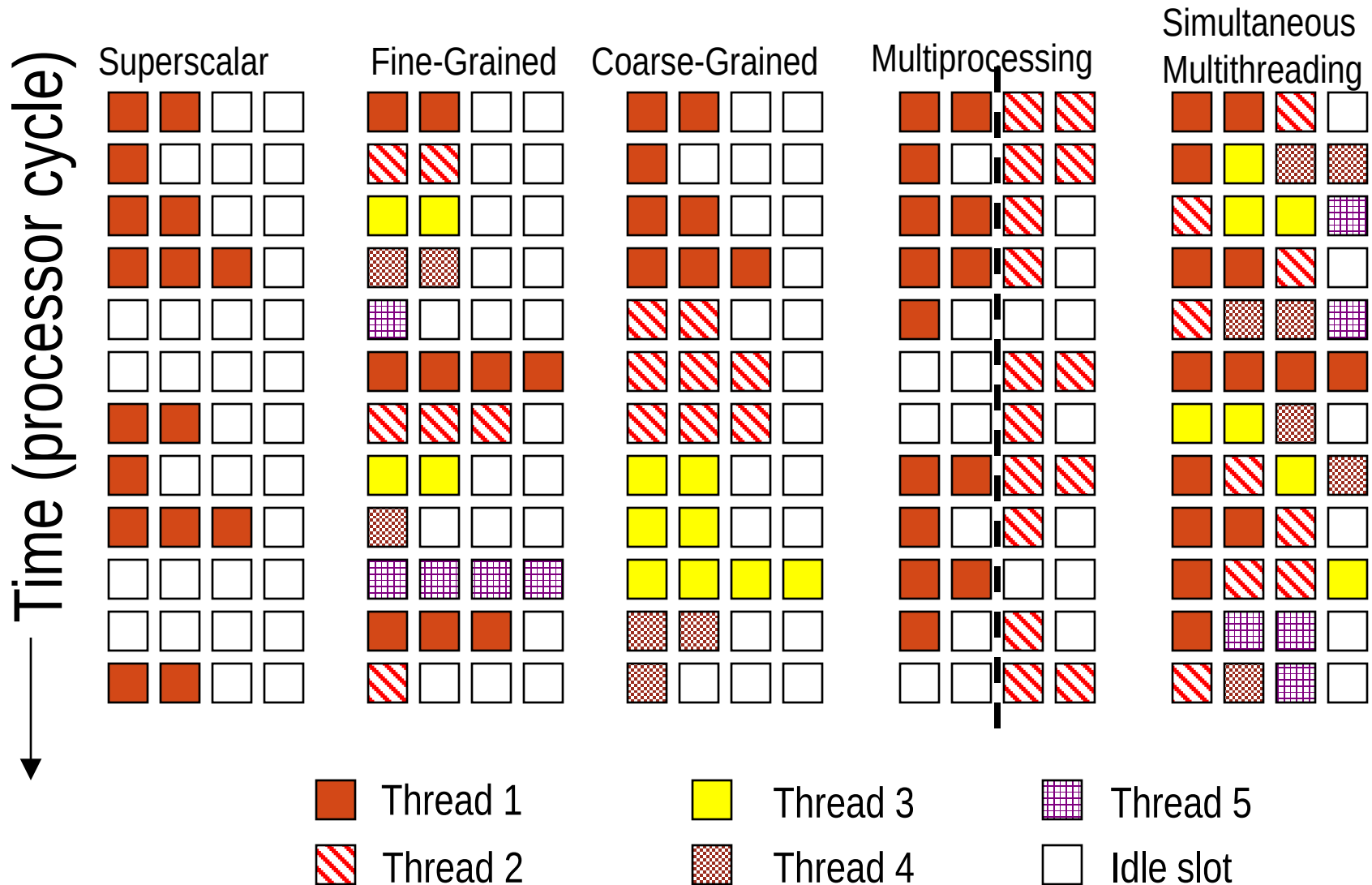
Simultaneous Multithreading (SMT) - I

- Simultaneous multithreading (SMT): insight that dynamically scheduled processor already has **many HW mechanisms** to support multithreading
 - **Large set of virtual registers** that can be used to hold the register sets of independent threads
 - **Register renaming** provides unique register identifiers, so instructions from multiple threads can be mixed in datapath
 - **Out-of-order completion** allows the threads to execute out of order
- **Just adding a per thread renaming table and keeping separate PCs**
 - Independent commitment can be supported by logically keeping a **separate reorder buffer** for each thread

Simultaneous Multithreading - II

- Add multiple contexts and fetch engines to wide out-of-order superscalar processor
 - [Tullsen, Eggers, Levy, UW, 1995]
- OOO instruction window already **has most of the circuitry required** to schedule from multiple threads
- Any single thread can utilize whole machine

Multithreaded Categories



Design Challenges in SMT

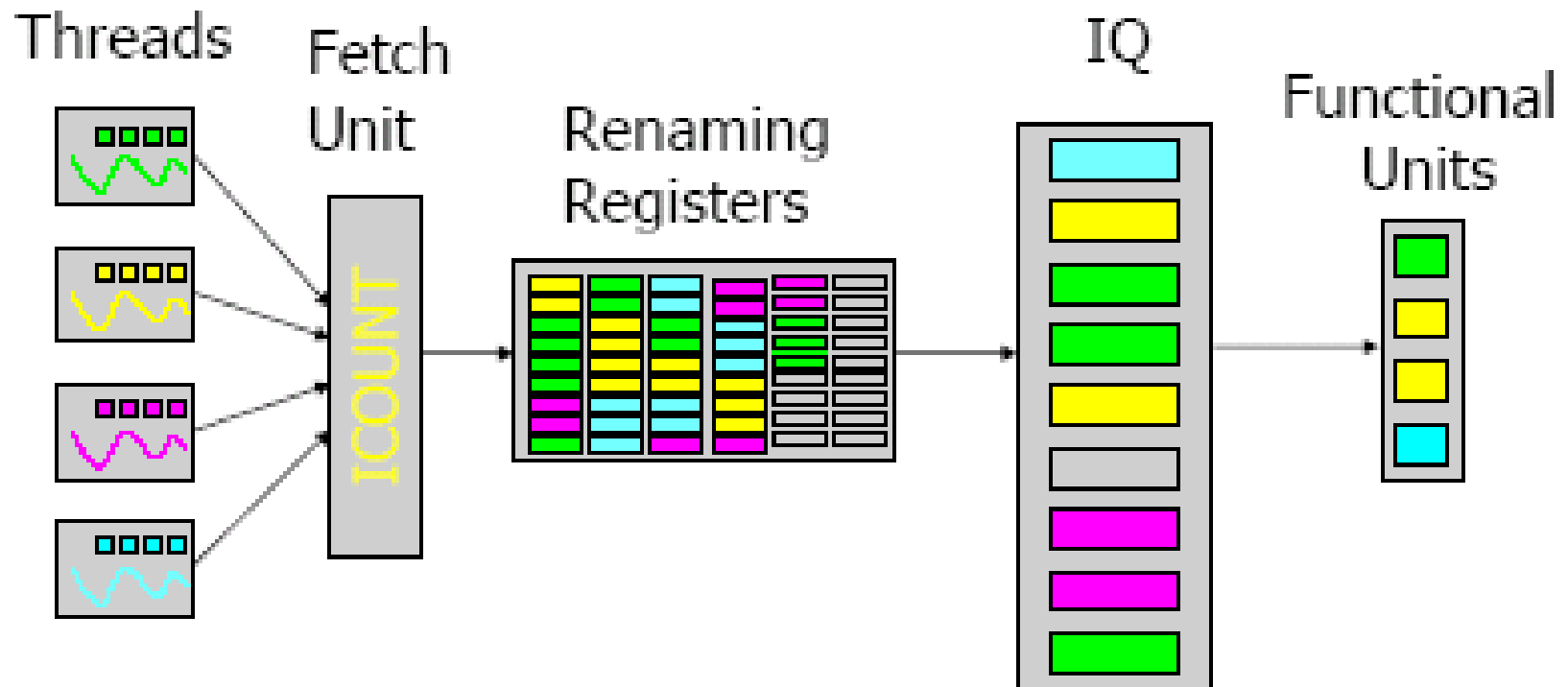
- Since **SMT makes sense only with fine-grained** implementation, what is its impact on single thread performance?
 - Unfortunately, with a preferred thread, the processor is likely to sacrifice some throughput, when preferred thread stalls
- Larger register file needed to hold multiple contexts
- Not affecting clock cycle time, especially in
 - **Instruction issue** - more candidate instructions need to be considered
 - **Instruction completion** - choosing which instructions to commit may be challenging
- Ensuring that cache and TLB conflicts generated by SMT do not degrade performance

From Superscalar to Multithreading

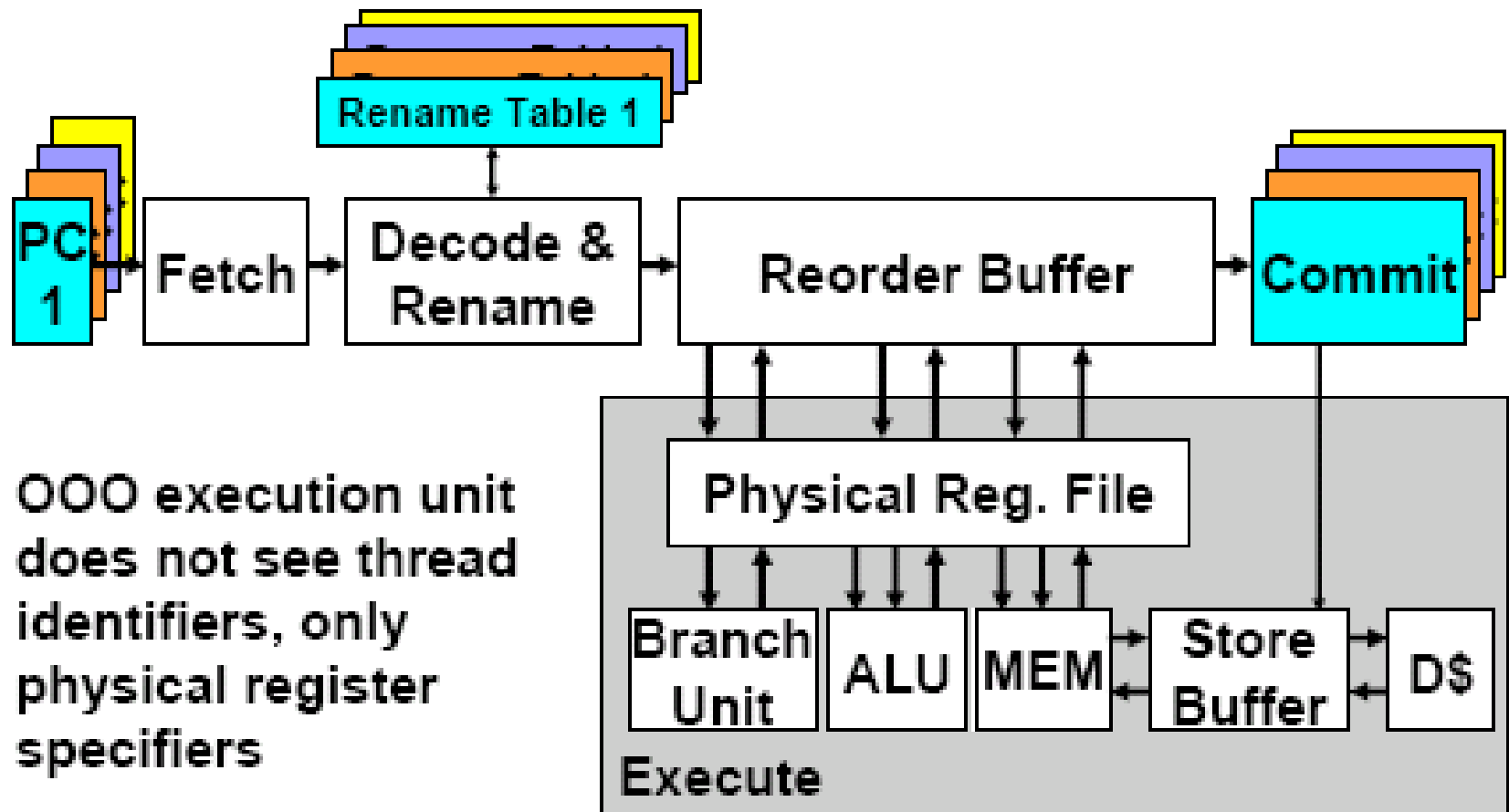
- Implications
 - per-thread program counters
 - per-thread return stacks
 - per-thread bookkeeping for instruction retirement, trap & instruction dispatch, queue flush
 - thread identifiers, e.g., with
 - BTB (Branch Target Buffer) &
 - TLB (Translation Lookaside Buffer) entries

From Superscalar to SMT

- SMT is an out-of-order superscalar extended with hardware to support multiple executing threads



Simultaneous Multithreaded Processor



Intel Pentium-4 Xeon Processor

- **Hyperthreading == SMT**
- Dual physical processors, each 2-way SMT
- **Logical processors** share all resources of the physical processor
 - Caches, execution units, branch predictors
- Die area overhead of **hyperthreading** ~5 %
- When one **logical processor is stalled**, the other can make progress (**Coarse-grained multithreading**)
- A processor running only one active software thread to **run at the same speed with or without hyperthreading**

Intel Pentium-4 Xeon Processor

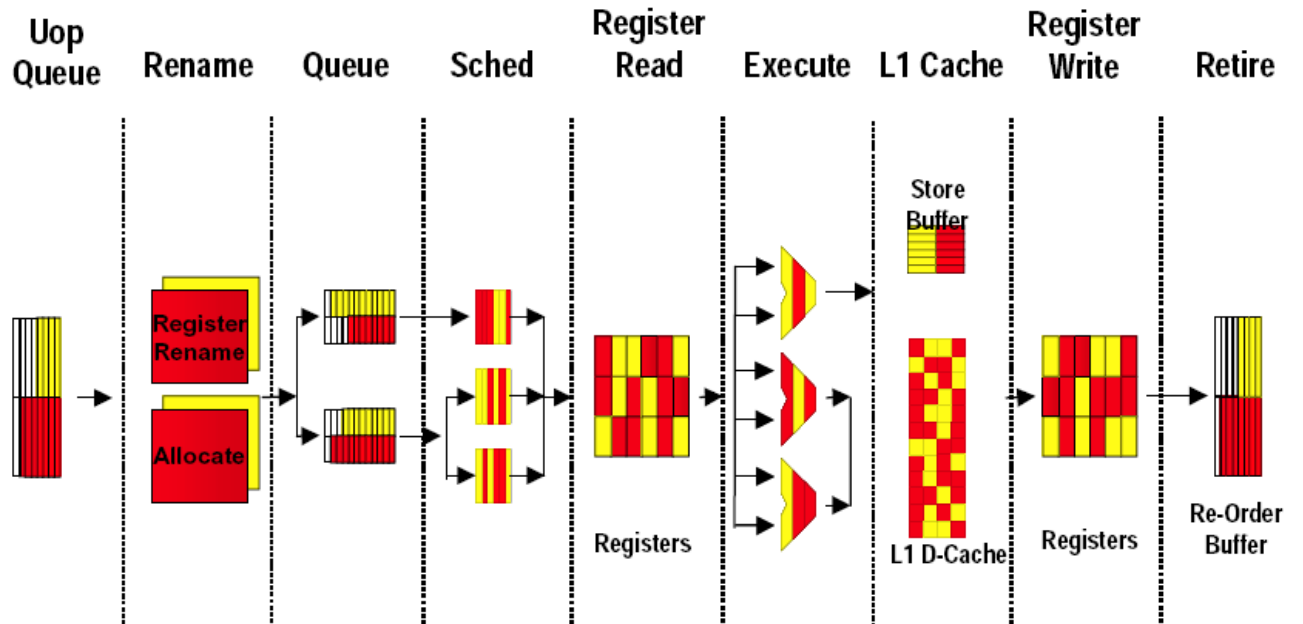


Figure 6: Out-of-order execution engine detailed pipeline

1. The allocator logic takes uops from the uop queue and allocates many of the key machine buffers needed to execute each uop. Each logical processor can use up to a maximum of 63 re-order buffer entries, 24 load buffers, and 12 store buffer entries.
2. **The two sets of queues** are called the **memory instruction queue** and the **general instruction queue**, respectively.
3. Five uop schedulers are used to schedule different types of uops for the various execution units. Collectively, they can **dispatch up to six uops each clock cycle**.

Intel Xeon Performance

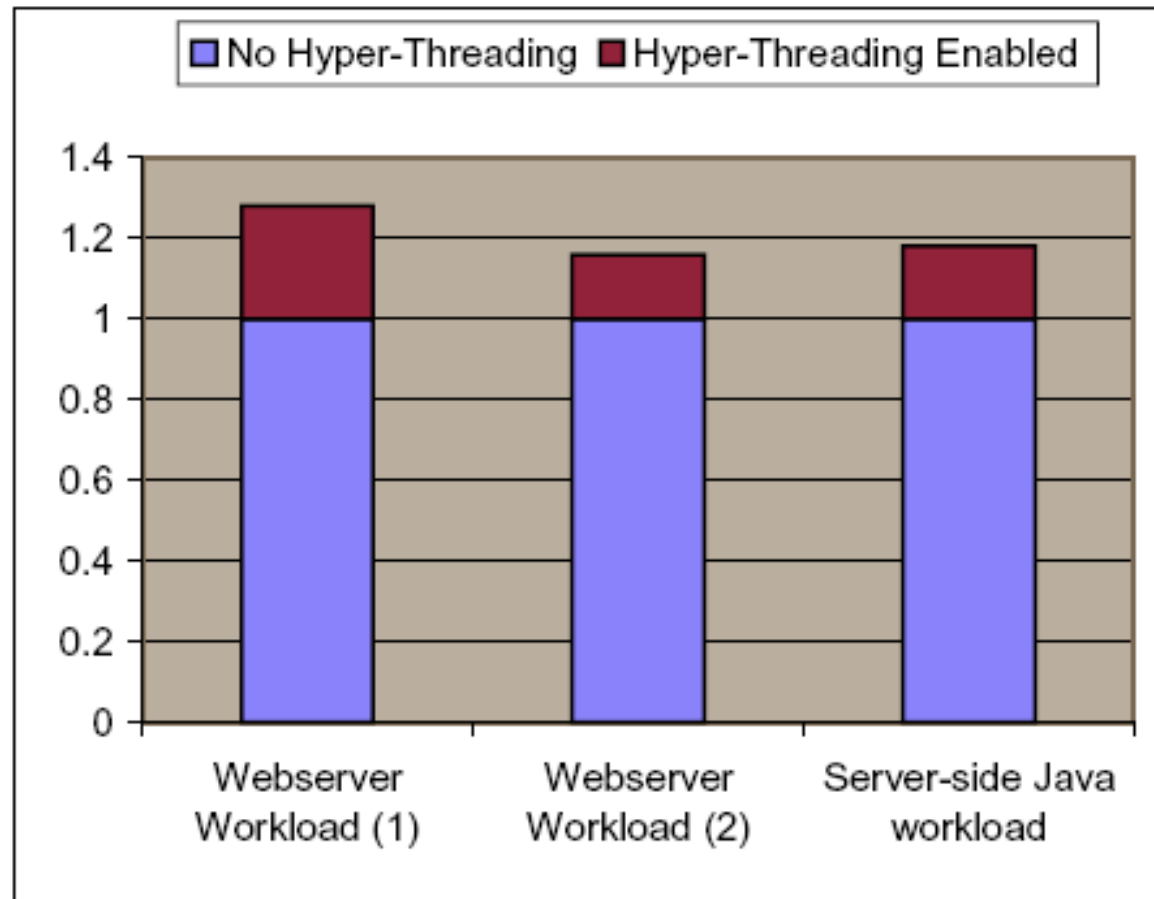


Figure 9: Web server benchmark performance

In Summary ...

- Limits to ILP (power efficiency, compilers, dependencies ...) seem to limit to 3 to 6 issue for practical options
- Explicitly parallel (Data level parallelism or Thread level parallelism) is next step
- Coarse grain vs. Fine grained multithreading
 - Only on big stall vs. every clock cycle
- Simultaneous Multithreading if fine grained multithreading based on OOO superscalar microarchitecture
 - Instead of replicating registers, reuse rename registers
- Itanium/VLIW is not a breakthrough in ILP
- Balance of ILP and TLP decided in marketplace

What is Multiprocessing

- Parallelism at the Instruction Level is limited because of data dependency => Speed up is limited!!
- Abundant availability of program level parallelism, like Do I = 1000, **Loop Level Parallelism**. How about employing multiple processors to execute the loops => **Parallel processing or Multiprocessing**
- With billion transistors on a chip, we can put a few CPUs in one chip => **On-Chip multiprocessor**

On-Chip Multiprocessors

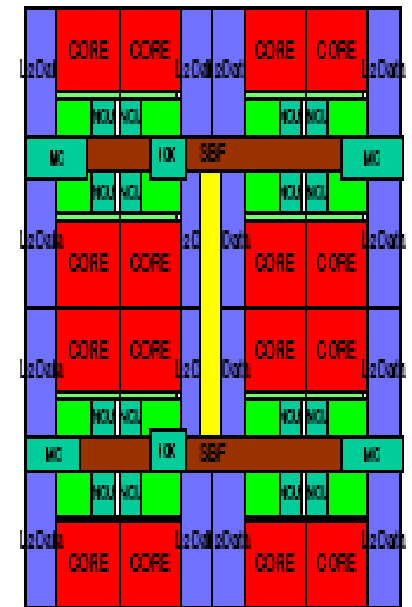
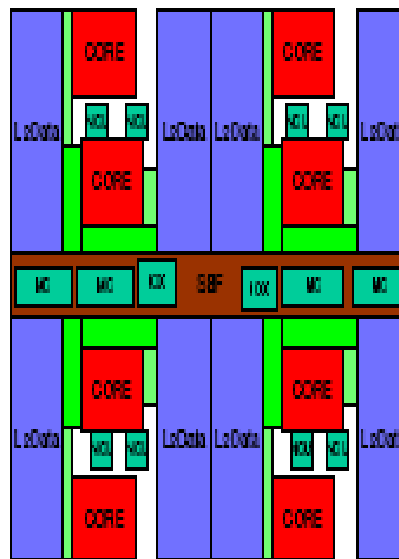
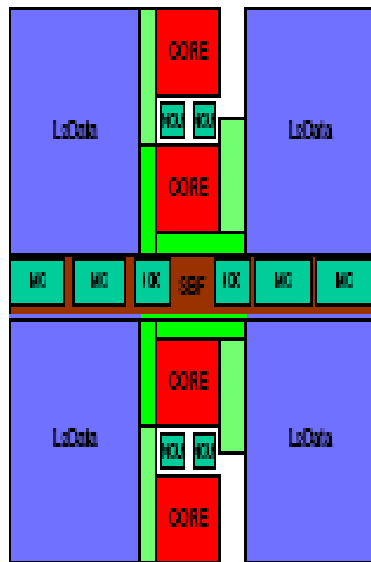


Figure 3. Floorplans for 4, 8 and 16 core processors